

System e-commerce sklepu odzieżowego

Podsumowanie i Wdrożenie Projektu

Kacper Gumulak & Damian Judka

Cel i Założenia Architektoniczne

Architektura "Thick Database"

- Głównym założeniem było udowodnienie, że **relacyjna baza danych** potrafi znacznie więcej niż tylko składowanie tabel.
- Zbudowaliśmy system oparty na silniku **MariaDB 11 (InnoDB)**.
- Większość krytycznej logiki biznesowej zamknięto bezpiecznie po stronie serwera SQL, gwarantując niezawodność reguł **ACID**.
- Frontend to lekki, zwinny interfejs w języku **Python (Streamlit)** działający jedynie jako bezstanowy pośrednik.



Model Komponentów i Wdrożenie



Infrastructure as Code

Środowisko jest w pełni replikowalne i wdrożone za pomocą jednego pliku [docker-compose.yml](#). Uruchomienie infrastruktury trwa poniżej 2 minut.



Izolacja Zmiennych (.env)

Twarde reguły bezpieczeństwa. Hasła root'a i poświadczenia wyciągnięte z kodu źródłowego do zignorowanego lokalnego pliku **.env**.



Konteneryzacja Docker

Streamlit i MariaDB pracują we własnej, zamkniętej sieci. Baza nie jest wyeksponowana na zewnątrz, chroniąc przed bezpośrednim atakiem.

Serce Systemu (Logika SQL)

Potężny backend

Cały fundament e-commerce to potężny, natywny skrypt bazy danych liczący **ok. 2700 linii kodu**, wdrażany automatycznie przy starcie.

- ✓ **Event Sourcing** w magazynie – wyliczanie dostępności ze zdarzeń, likwidacja problemu *oversellingu*.
- ✓ **Złożone Procedury** – np. `pay_order()` aktualizująca status i generująca fakturę w jednej transakcji.
- ✓ **Nested Set** – optymalna, drzewiasta struktura dla zagnieżdżonych kategorii ubrań.

```
469 CREATE PROCEDURE `pay_order` (IN `id_order` BIGINT(20) UNSIGNED) BEGIN
470
471     DECLARE transaction_id VARCHAR(50);
472     DECLARE total_net_value INT;
473     DECLARE total_gross_value INT;
474     DECLARE error_message TEXT;
475     DECLARE id_invoice BIGINT;
476     DECLARE id_cart BIGINT;
477
478     DECLARE EXIT HANDLER FOR SQLEXCEPTION
479     BEGIN
480         GET DIAGNOSTICS CONDITION 1
481             error_message = MESSAGE_TEXT;
482         ROLLBACK;
483         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = error_message;
484     END;
485
486     SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
487
488     START TRANSACTION;
489
490     -- Wygenerowanie ID transakcji
491     SET transaction_id = generate_transaction_id(id_order);
492
493     -- Obliczenie całkowitej wartości netto zamówienia
494     SET total_net_value = calculate_cart_total_value(id_order, FALSE);
495
496     -- Obliczenie całkowitej wartości brutto zamówienia
497     SET total_gross_value = calculate_cart_total_value(id_order, TRUE);
```

Quality Assurance i Testy Automatyczne

8/8
PASSED

Zautomatyzowany audyt w środowisku Python (Pytest)

Przeprowadziliśmy bezkompromisowe testowanie rdzenia bazy danych poprzez natywny sterownik PyMySQL w dedykowanych sesjach, omijając UI.

-  **4 testy Unitowe:** hasła, unikalność, walidacje ilościowe.
-  **2 testy Integracyjne:** blokada pustych koszyków i oversellingu.
-  **2 testy E2E (End-to-End):** pełna symulacja życia obiektu od koszyka aż po zwrot towaru w logach.

Scenariusz Pokazu: Live Demo

2. Płatność

Symulacja Checkoutu.
Procedura `pay_order()`
autoryzuje transakcję w
bazie.

4. Zwrot

Logistyka Admina. Zwrot
towaru generuje dynamiczny
wpis w `stock_events`.

1. Koszyk

Przegląd asortymentu w UI
klienta, walidacja
dostępności i dodanie
odzieży do koszyka.

3. Faktura

Natychmiastowe, ukryte
wygenerowanie dokumentów
księgowych w tej samej
transakcji.

Wnioski i Hardening (DevOps)

Błędy i konflikty wyłapane w testach:

- **Error 1644:** Konflikt transakcyjny (Python utrzymywał aktywną transakcję, blokując READ COMMITTED w procedurze SQL).
- **Integrity Error 1451:** Zablokowane usuwanie konta klienta z powodu resztek w koszyku.
- **Collation Mismatch:** Błąd kodowania w Dockerze uca1400_ai_ci vs general_ci podczas instrukcji rzutujących znakami.

Wdrożone Zabezpieczenia:

```
# Autocommit w sterowniku db_connector
connection = pymysql.connect(..., autocommit=True)
```

```
# Poprawka schematu bazy danych      ALTER TABLE
`cart` ADD CONSTRAINT FOREIGN KEY ( ... )
ON DELETE CASCADE
```

```
# Parametryzacja serwera w Docker Compose  command:
- --character-set-server=utf8mb4 - --collation-
server=utf8mb4_general_ci
```

Dziękujemy za uwagę

Czy mają Państwo pytania dotyczące architektury lub działania systemu?